CivicSuite — User Manual

This is the orientation manual for the CivicSuite umbrella repo. It is written in three parts so different audiences can read only what's relevant to them:

1. **For municipal decision-makers** — non-technical overview of what CivicSuite is, what's available today, and what the licensing and sovereignty posture means for your city. 2. **For developers and IT staff** — how the umbrella works, where each module lives, and how to evaluate or contribute. 3. **Architecture reference** — diagrams and dependency rules.

A glossary at the end defines every technical term used.

Part 1 — For municipal decision-makers

What is CivicSuite?

CivicSuite is an **open-source product family** for municipal records and civic operations. It's not one program — it's a planned collection of modules a city can install one at a time, on its own hardware, on its own schedule. Cities never have to send data to a vendor's cloud, never pay per user, and can read or modify the source code anytime.

Today, only one module is shipping. The rest are either in early-platform stage or planned. We say so plainly below — no roadmap inflation, no vaporware.

What's available today (as of 2026-04-25)

- **civicrecords-ai v1.4.0** — a working, shipping module for managing public records / FOIA requests. Cities can install this today. Repo: <https://github.com/scottconverse/civicrecords-ai>. - **civiccore v0.2.0** — the shared "platform" package that every module uses. It is what the records module is built on. As of v0.2.0 it includes a shared LLM (large-language-model) abstraction layer. It is not a product on its own; you only "install" it as a dependency of a module. Repo: <https://github.com/CivicSuite/civiccore>.

What's planned but not started

- **civicclerk** — meetings, agendas, packets, minutes, voting, and sunshine-law compliance. Spec drafted, no code yet. - **civiczone** — zoning code and parcel-aware planner workflows. Spec drafted, no code yet. - Twenty-plus additional modules across seven tiers — see the [module catalog](specs/01_catalog.md).

If you don't see a module on this list with a version number, **it does not exist as code yet**. Specs are not products.

What this means for your city

- **No vendor lock-in.** You install on your own hardware. If a maintainer disappears, the code is yours. The code license (Apache 2.0) and documentation license (CC BY 4.0) both allow you to fork, modify, and continue using the software indefinitely. - **No cloud dependency.** Modules are designed to run with no outbound network calls. The default LLM (Gemma 4, served locally via [Ollama](https://ollama.com/)) runs on the city's own machine. - **No per-seat billing.** You add as many users as you need at no marginal cost. - **You evaluate one module at a time.** Don't install the suite — install records-ai, see if it solves a real problem in your clerk's office, and decide whether to install another module later. - **You are responsible for hosting and operations.** This is not a SaaS product. Your IT staff (or a contractor) operates the server. The user manual for each module documents what's required.

Glossary (Part 1)

- **FOIA** — Freedom of Information Act; the federal and state laws that govern public records requests. - **LLM** — Large language model; the AI technology used for things like classifying requests, suggesting redactions, and drafting responses. - **Open source** — software whose code is published publicly under a license that allows anyone to read, modify, or redistribute it. - **Sovereign deployment** — software that runs entirely on hardware the city controls, with no required outbound calls to a vendor.

Part 2 — For developers and IT staff

How the umbrella works

The `civicsuite` repo (this one) is **documentation-only**. It contains:

- The [Charter](CHARTER.md) — the project's founding document. - The [Consistency reference](CONSISTENCY.md) — the audit table of every cross-reference and count, treated as the truth-source. - Specs for the catalog, civiccore extraction, civicclerk, and civiczone in `specs/`. - ADRs (Architecture Decision Records) under `docs/architecture/`. - The [compatibility matrix](docs/compatibility/index.md) — which civicrecords-ai version pins to which civiccore version. - The [GitHub Pages landing site](docs/index.html).

There is no runtime code in this repo. Each module lives in its own repo.

Module repos

| Module | Repo | Status | |---|---|---| | civicrecords-ai | <https://github.com/scottconverse/civicrecords-ai> | Shipping v1.4.0. Will be transferred to the `CivicSuite` GitHub org at a future date — until then, the canonical home is `scottconverse/civicrecords-ai`. | | civiccore | <https://github.com/CivicSuite/civiccore> | Shipping v0.2.0. Phase 2 (LLM module) just landed. | | civicclerk, civiczone, etc. | not created yet | Specs only. |

Dependency direction

- Modules depend on civiccore. **Civiccore never depends on a module.** - This is enforced in CI in the civiccore repo and is documented in the [extraction spec](specs/02_CivicCore.md) section 5.2. - Modules

are pinned to a civiccore version in their package manifest. The compatibility matrix is the truth-source for which pin is required for which module version.

How civiccore is consumed

A module like `civicrecords-ai` declares civiccore as a dependency in its `pyproject.toml`:

```
dependencies = [
  "civiccore==0.2.0",
  ...
]
```

When a city installs records-ai, civiccore is pulled in as a wheel. There is no monorepo, no submodule, no vendored copy. Civiccore is published to PyPI (or, until publication, distributed as a release wheel from its GitHub repo).

Evaluating a module

1. Read the module's `README.md` (start with the front-door pitch). 2. Read its `USER-MANUAL.md` for a non-technical operations walkthrough. 3. Check `CHANGELOG.md` to see release cadence and recent breaking changes. 4. Spin up a local install following the module's `CONTRIBUTING.md` setup steps. The umbrella does not host setup instructions for individual modules — those live with the module. 5. If the module's tests pass on a clean clone, the project meets the umbrella's minimum bar.

Contributing

- **Suite-wide questions, ADRs, governance, roadmap, compatibility matrix** → contribute here. See CONTRIBUTING.md. - **Module bugs, module features, module docs** → contribute on the module's repo. The bug-routing decision tree in CONTRIBUTING.md tells you where each kind of bug goes.

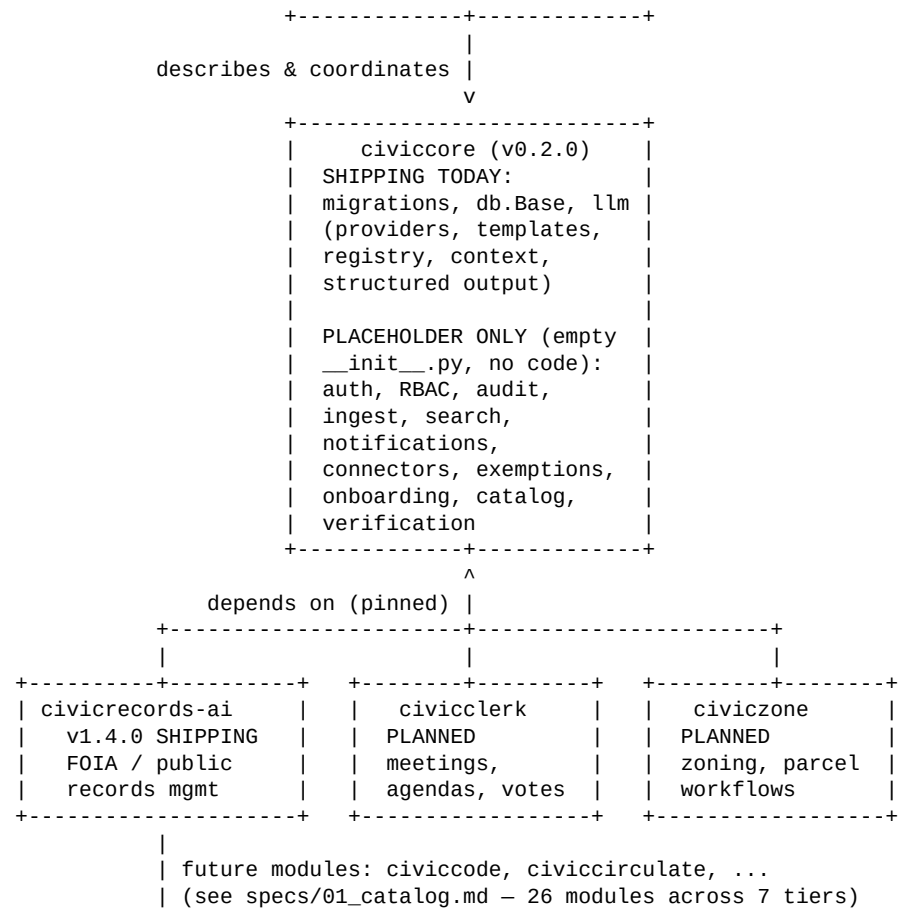
Release coordination

When a module ships a new version, the compatibility matrix on this umbrella must be updated. Cross-cutting changes (e.g. civiccore breaks an API that records-ai uses) require a paired release: civiccore ships first, then records-ai ships pinned to the new civiccore. The release pairing is documented in the matrix.

Part 3 — Architecture reference

Suite topology (textual diagram)

```
+---------------------------+
|   civicsuite (umbrella)   |
|   docs, ADRs, governance, |
|   compatibility matrix    |
```



Dependency rule

Modules import from civiccore. Civiccore never imports from modules. This is the core architectural constraint of the suite. It is enforced by a lint rule in CI in the civiccore repo. Violating it would silently couple modules to each other through civiccore and destroy the modular install promise.

Migration / upgrade order

When civiccore ships a backward-compatible version (PATCH or MINOR):

1. civiccore releases the new version. 2. The umbrella's compatibility matrix is updated. 3. Module maintainers update their pin opportunistically (no forced upgrade).

When civiccore ships a breaking version (MAJOR):

1. The breaking change is announced in advance via an ADR. 2. civiccore releases the new MAJOR version. 3. Each module that consumes civiccore ships a paired MAJOR release pinned to the new civiccore version. 4. The compatibility matrix is updated to show both the old (still supported) and new pairing during a transition window.

Glossary (Part 3)

- **ADR** — Architecture Decision Record; a short, dated document recording a significant architectural decision and its rationale. - **Pinned version** — a specific exact version a module requires of civiccore (e.g. `==0.2.0`), as opposed to a range. - **Wheel** — the standard Python package distribution format. civiccore is published as a wheel. - **CI** — continuous integration; the automated test pipeline that runs on every commit and pull request. - **Monorepo** — a single repository containing multiple projects. CivicSuite is deliberately *not* a monorepo; each module has its own repo so cities can install modules independently.

What to do when something goes wrong

| Symptom | Where to look | |---|---| | Module won't install | The module's `README.md` and `CONTRIBUTING.md` setup steps. Most install errors are resolved by matching the documented Python/PostgreSQL/Redis versions. | | civiccore pin mismatch | The compatibility matrix at docs/compatibility/index.md. | | You found a bug, don't know where to file | The bug-routing decision tree in CONTRIBUTING.md. | | Security issue | SECURITY.md. | | You have a general question | SUPPORT.md. |